



Department of Computer Science & Engineering

Course Code: CSE 430

Course Title: Compiler Design Lab

Project Name: Mini_Compiler_Project

Date of Submission: 18/10/2025

Submitted To:

Md.Shafayetul Haque
Lecturer,
Department of CSE, UAP

Submitted By:

Swapna Khanam
Reg No: 21201082
SEC: B2

Abstract:

This Mini Compiler Project demonstrates the complete compilation process for a simplified programming language. The project integrates all major phases of a compiler: lexical analysis, syntax analysis, abstract syntax tree (AST) generation, intermediate code generation (ICG), code optimization, and assembly code generation. Tools used include Flex, Bison (Yacc), GCC, and Python. Each task has been implemented and verified successfully with appropriate input/output examples.

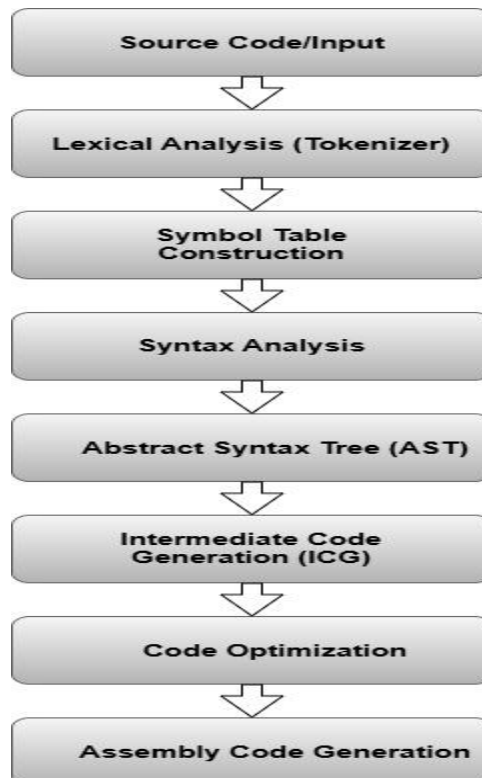
Introduction:

The objective of this project is to design and implement a small compiler that can analyze, process, and translate source code into optimized intermediate and assembly representations. Each compilation phase has been developed independently to ensure modularity and clarity. This project was implemented in the C programming language using Lex and Yacc for the compiler front-end, and Python for code optimization and assembly generation.

System Design:

The compiler consists of seven sequential tasks, each responsible for a specific compilation phase:

- Task 1 – Lexical Analysis (Tokenizer)
- Task 2 – Symbol Table Construction
- Task 3 – Syntax Analysis
- Task 4 – Abstract Syntax Tree (AST)
- Task 5 – Intermediate Code Generation (ICG)
- Task 6 – Code Optimization
- Task 7 – Assembly Code Generation



Project Structure:

```
Mini_Compiler_Project/
├── Mini_Compiler_Project/
│   ├── run_and_show.sh
│   ├── Makefile
│   ├── outputs/
│   │   ├── task1_lexer_output.txt
│   │   ├── task2_symbol_table_output.txt
│   │   ├── task3_syntax_output.txt
│   │   ├── task4_ast_output.txt
│   │   ├── task5_icg_output.txt
│   │   ├── task6_optimization_output.txt
│   │   └── task7_assembly_output.txt
│   ├── task1_lexer/
│   │   ├── lex.yy.c
│   │   ├── token.l
│   │   ├── input.c
│   │   ├── lexer
│   │   └── README.txt
│   ├── task2_symbol_table/
│   │   ├── lex.yy.c
│   │   ├── symbol_table
│   │   ├── symbol.l
│   │   ├── README.txt
│   │   └── input.c
│   ├── task3_syntax_analysis/
│   │   ├── syntax.tab.h
│   │   ├── syntax.tab.c
│   │   ├── lex.yy.c
│   │   ├── parser
│   │   ├── syntax.l
│   │   ├── syntax.y
│   │   ├── README.txt
│   │   └── input.c
│   ├── task4_ast/
│   │   ├── tree.tab.h
│   │   ├── tree.tab.c
│   │   ├── lex.yy.c
│   │   ├── ast
│   │   ├── tree.l
│   │   ├── tree.y
│   │   ├── README.txt
│   │   └── input.c
│   ├── task5_icg/
│   │   ├── lex.yy.c
│   │   ├── y.tab.h
│   │   ├── y.tab.c
│   │   ├── tac
│   │   ├── icg
│   │   ├── tac.l
│   │   ├── tac.y
│   │   ├── README.txt
│   │   └── input.c
│   ├── task6_optimization/
│   │   ├── optimized.txt
│   │   ├── input.txt
│   │   ├── icg_optimize.py
│   │   └── README.txt
│   └── task7_assembly/
│       ├── input22.s
│       ├── input22.txt
│       ├── assemble.py
│       └── README.txt
```

Lexical Analysis (Tokenizer)

Design:

- Use Flex (lex) to write regular expressions for each token class.
- Token types: KEYWORD, IDENTIFIER, INT_LITERAL, FLOAT_LITERAL, STRING_LITERAL, OPERATOR, PUNCTUATION, COMMENT, WHITESPACE.
- Implement actions that return tokens with semantic values (e.g., yylval for parser).
- Use start conditions for handling contexts (e.g., string literals, comments).

Symbol Table Construction

Design:

- Use a hash table for fast identifier lookup (average $O(1)$).
- Each symbol table entry contains: name, type, scope level, data type, size, line declared, attributes (isFunction, parameters list), memory location/address.
- Scope management: implement a stack of symbol tables to support nested scopes (e.g., functions, compound statements).

Syntax Analysis (Parser)

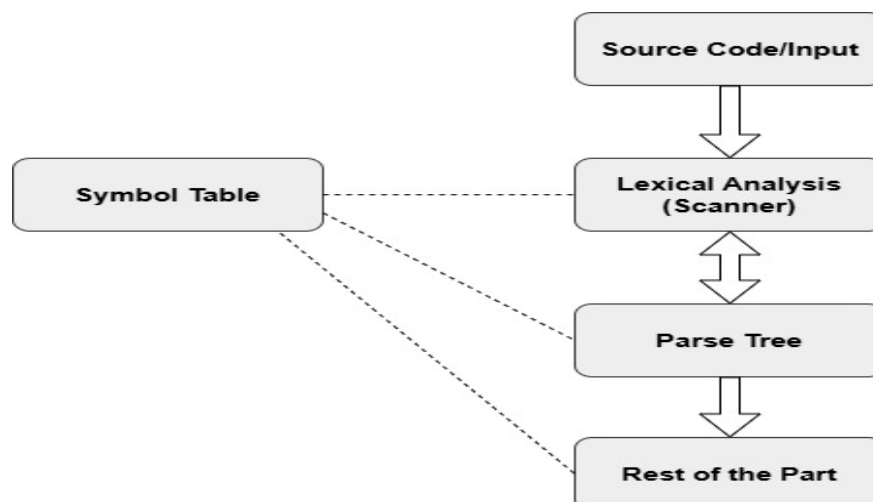
Design:

- Use Bison (Yacc) to define an LALR(1) grammar for the language constructs (expressions, statements, control flow, function definitions).
- Grammar is divided into productions for declarations, statements, expressions, and program structure.
- Employ semantic actions in grammar rules to build AST nodes or to perform immediate semantic checks (e.g., type checking for simple cases).
- Error recovery: implement simple error productions and bison error token to recover and continue parsing.

Abstract Syntax Tree (AST)

Design:

- AST node types: Program, FunctionDecl, VarDecl, CompoundStmt, IfStmt, WhileStmt, ForStmt, ReturnStmt, Expr (BinaryOp, UnaryOp, Literal, Identifier), CallExpr.
- Each node contains: node type, value or operator, type information (if available), children list or left/right pointers, source location for debugging.



Intermediate Code Generation (ICG)

Design:

- Use a linear list of quadruples or three-address instructions: (op, arg1, arg2, result).
- Implement temporary variable management (t1, t2, ...) and a counter to name temporaries.
- Translate high-level constructs into sequences of TAC: expressions become sequences computing into temporaries; control flow uses labels and conditional/unconditional jumps.

Code Optimization

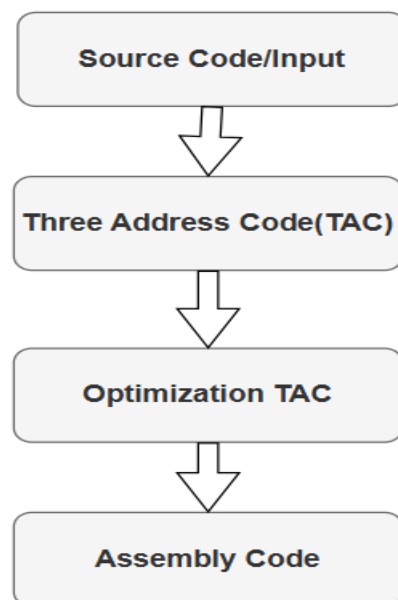
Design:

- Implement simple local/global optimizations using Python scripts that process the TAC file.
- Optimization techniques included:
 - Constant Folding: evaluate constant expressions at compile time (e.g., $2+3 \rightarrow 5$).
 - Constant Propagation: replace variables that have constant values with the constants.
 - Dead Code Elimination: remove assignments whose results are never used.
 - (Optional) Common Subexpression Elimination: detect repeated expressions and reuse temporaries.

Assembly Code Generation

Design:

- Use a mapping from TAC operations to assembly instruction sequences.
- Implement simple register allocation using a limited register pool with a spill-to-memory strategy (hash map to track variable locations).
- Manage function calls: create stack frames, save/restore callee-saved registers, handle parameter passing (stack or registers), and return values



Implementation:

The following sections describe the detailed implementation and execution steps for each compiler phase. Commands to run each phase are listed.

Task 1 - Lexical Analysis(Tokenizer)

Tools used: Flex (Lex) and GCC

Language: C

Description: Tokenizes the input source code into identifiers,keywords,operators.etc

Command:

```
cd task1_lexer
```

```
lex token.l
```

```
gcc lex.yy.c -o lexer
```

```
./lexer < input.c
```

Task 2 - Symbol Table Construction

Tools used: Flex and GCC

Language: C

Description: Builds a symbol table from source code containing identifiers and their attributes

Command:

```
Cd ../task2_symbol_table
```

```
lex symbol.l
```

```
gcc lex.yy.c -o symbol_table
```

```
./symbol_table < input.c
```

Task 3 - Syntax Analysis

Tools used: Flex, Bison (Yacc),Flex, GCC

Language: C

Description: Parses the input tokens according to grammar rules and detects syntax errors

Command:

```
cd ../task3_syntax_analysis
```

```
bison -d syntax.y
```

```
flex syntax.l
```

```
gcc syntax.tab.c lex.yy.c -o parser -lfl
```

```
./parser < input.c
```

Task 4 - Abstract Syntax Tree (AST)

Tools used: Flex, Bison, GCC

Language: C

Description: Constructs an AST for parsed expressions showing hierarchical structure

Command:

```
Cd ../task4_ast
```

```
bison -d tree.y
```

```
flex tree.l
gcc tree.tab.c lex.yy.c -o ast -lfl
./ast < input.c
```

Task 5 - Intermediate Code Generation

Tools used: Flex, Bison, GCC
Language: C
Description: Generates three-address code (TAC) from AST
Command:
Cd ../task5_icg
lex tac.l
yacc -d tac.y
gcc lex.yy.c y.tab.c -lfl -o tac
./tac < input.c

Task 6 - Code Optimization

Tools used: Python
Language: Python
Description: Optimizes three-address code by removing redundancies and simplifying expressions
Command:
Cd ../task6_optimization
python3 icg_optimize.py input.txt ..print

Task 7 - Assembly Code Generation

Tools used: Python
Language: Python
Description: Converts optimized intermediate code to target assembly language
Command:
cd ../task7_assembly
python3 assemble.py input22.txt
Note: Assembly code will be stored in filename.s: input22.s

Full Code Run:

File: run_and_show.sh
Command:
cd~/Documents/Mini_Compiler_Project
Chmod +x run_and_show.sh
./run_and_show.sh

Results:

Each compiler phase was tested with sample inputs. Outputs are stored in the outputs/ directory:

Task1_lexer_output.txt

Task2_symbol_table_output.txt

Task3_syntax_output.txt

Task4_ast_output.txt

Task5_icg_output.txt

Task6_optimizatuin_output.txt

Task7_assembly_output.txt

Task 1 - Lexical Analysis(Tokenizer)

Input:

```
task1_lexer > C input.c
1  int a;
2  int b;
3  a = 5;
4  b = 0;
5  for (a = 0; a < 5; a = a + 1) {
6      if (a % 2 == 0) {
7          b = b + a;
8      } else {
9          b = b + 1;
10     }
11 }
12 while (b > 0) {
13     b = b - 1;
14 }
15
```

Output:

```
* swapna@ubuntu:~/Documents/Mini_Compiler_Project$ cd task1_lexer
* swapna@ubuntu:~/Documents/Mini_Compiler_Project/task1_lexer$ lex token.l
* swapna@ubuntu:~/Documents/Mini_Compiler_Project/task1_lexer$ gcc lex.yy.c -o lexer
* swapna@ubuntu:~/Documents/Mini_Compiler_Project/task1_lexer$ ./lexer < input.c
1 int --> KEYWORD

1 a --> IDENTIFIER

1 ; --> PUNCTUATION

2 int --> KEYWORD

2 b --> IDENTIFIER

2 ; --> PUNCTUATION

3 a --> IDENTIFIER

3 = --> ASSIGN_OP

3 5 --> CONST

3 ; --> PUNCTUATION

4 b --> IDENTIFIER

4 = --> ASSIGN_OP

4 0 --> CONST

4 ; --> PUNCTUATION

5 for --> KEYWORD

5 ( --> PARENTHESIS

5 a --> IDENTIFIER

5 = --> ASSIGN_OP

5 0 --> CONST

5 ; --> PUNCTUATION

5 a --> IDENTIFIER

5 < --> REL_OP

5 5 --> CONST

5 ; --> PUNCTUATION

5 a --> IDENTIFIER

5 = --> ASSIGN_OP

5 a --> IDENTIFIER

5 + --> ARITH_OP

5 1 --> CONST

5 ) --> PARENTHESIS

5 { --> PARENTHESIS

6 if --> KEYWORD

6 ( --> PARENTHESIS

6 a --> IDENTIFIER

6 % --> ARITH_OP

6 2 --> CONST

6 == --> REL_OP

6 0 --> CONST

6 ) --> PARENTHESIS

6 { --> PARENTHESIS

7 b --> IDENTIFIER

7 = --> ASSIGN_OP

7 b --> IDENTIFIER

7 + --> ARITH_OP

7 a --> IDENTIFIER

7 ; --> PUNCTUATION

8 } --> PARENTHESIS

8 else --> KEYWORD

8 { --> PARENTHESIS

9 b --> IDENTIFIER

9 = --> ASSIGN_OP

9 b --> IDENTIFIER

9 + --> ARITH_OP

9 1 --> CONST

9 ; --> PUNCTUATION

10 } --> PARENTHESIS

11 } --> PARENTHESIS

12 while --> KEYWORD

12 ( --> PARENTHESIS

12 b --> IDENTIFIER

12 > --> REL_OP

12 0 --> CONST

12 ) --> PARENTHESIS

12 { --> PARENTHESIS

13 b --> IDENTIFIER

13 = --> ASSIGN_OP

13 b --> IDENTIFIER

13 - --> ARITH_OP

13 1 --> CONST

13 ; --> PUNCTUATION

14 } --> PARENTHESIS
```

Task 2 - Symbol Table Construction

Input:

```
task2_symbol_table > C input.c
1  int a;
2  int b;
3  a = 5;
4  b = 0;
5  for (a = 0; a < 5; a = a + 1) {
6      if (a % 2 == 0) {
7          b = b + a;
8      } else {
9          b = b + 1;
10     }
11 }
12 while (b > 0) {
13     b = b - 1;
14 }
15
```

Output:

```
• swapna@ubuntu:~/Documents/Mini_Compiler_Project/task1_lexer$ cd ../task2_symbol_table
• swapna@ubuntu:~/Documents/Mini_Compiler_Project/task2_symbol_table$ lex symbol.l
• swapna@ubuntu:~/Documents/Mini_Compiler_Project/task2_symbol_table$ gcc lex.yy.c -o symbol_table
• swapna@ubuntu:~/Documents/Mini_Compiler_Project/task2_symbol_table$ ./symbol_table < input.c
```

TOKEN#	DATA TYPE	TOKEN TYPE	TOKEN VALUE	LINES	DIM	ADDRESS
1	int	KEYWORD	int	1, 2	1	9999999
2	int	IDENTIFIER	a	1, 3, 5, 6, 7	1	9999999
3	int	IDENTIFIER	b	2, 4, 7, 9, 12, 13	1	9999999
4	for	KEYWORD	for	5	1	9999999
5	if	KEYWORD	if	6	1	9999999
6	else	KEYWORD	else	8	1	9999999
7	while	KEYWORD	while	12	1	9999999

Task 3 - Syntax Analysis

Input:

```
task3_syntax_analysis > C input.c
1  int a;
2  int b;
3  a = 5;
4  b = 0;
5  for (a = 0; a < 5; a = a + 1) {
6      if (a % 2 == 0) {
7          b = b + a;
8      } else {
9          b = b + 1;
10     }
11 }
12 while (b > 0) {
13     b = b - 1;
14 }
15
```

Output:

```
• swapna@ubuntu:~/Documents/Mini_Compiler_Project/task3_syntax_analysis$ bison -d syntax.y
syntax.y: warning: 37 shift/reduce conflicts [-Wconflicts-sr]
syntax.y: note: rerun with option '-Wcounterexamples' to generate conflict counterexamples
• swapna@ubuntu:~/Documents/Mini_Compiler_Project/task3_syntax_analysis$ flex syntax.l
• swapna@ubuntu:~/Documents/Mini_Compiler_Project/task3_syntax_analysis$ gcc syntax.tab.c lex.yy.c -o parser -lfl
• swapna@ubuntu:~/Documents/Mini_Compiler_Project/task3_syntax_analysis$ ./parser < input.c

decl:int
id:a
semicolon
decl:int
id:b
semicolon
id:a
assignop:=
num:5
semicolon
id:b
assignop:=
num:0
semicolon
for
(
id:a
assignop:=
num:0
semicolon
id:a
compop:<
num:5
semicolon
id:a
assignop:=
id:a
addop:+
num:1
)
{
if
(
id:a
modop:%
num:2
compop:==
num:0
)
{
id:b
assignop:=
id:b
addop:+
num:1
semicolon
}
else
{
id:b
assignop:=
id:b
addop:+
num:1
semicolon
}
}
while
(
id:b
compop:>
num:0
)
{
id:b
assignop:=
id:b
subop:-
num:1
semicolon
}

Symbol Table
Symbol Scope dtype Value
a 1 int 0
b 1 int 0
```

Task 4 - Abstract Syntax Tree (AST)

Input:

```
task4_ast > c input.c
1 int a;
2 int b;
3 a = 5;
4 b = 0;
5 for (a = 0; a < 5; a = a + 1) {
6     if (a % 2 == 0) {
7         b = b + a;
8     } else {
9         b = b + 1;
10    }
11 }
12 while (b > 0) {
13     b = b - 1;
14 }
15
```

Output:

```
swagha@ubuntu:~/Documents/Mini_Compiler_Project/task3_syntax_analysis$ cd ../task4_ast
swagha@ubuntu:~/Documents/Mini_Compiler_Project/task4_ast$ bison -d tree.y
tree.y: warning: 25 shift/reduce conflicts [-Wconflicts-sr]
tree.y: note: rerun with option '-Wcounterexamples' to generate conflict counterexamples
swagha@ubuntu:~/Documents/Mini_Compiler_Project/task4_ast$ flex tree.l
swagha@ubuntu:~/Documents/Mini_Compiler_Project/task4_ast$ gcc tree.tab.c lex.yy.c -o ast -lfl
swagha@ubuntu:~/Documents/Mini_Compiler_Project/task4_ast$ ./ast < input.c
Parsing and building AST...
stat_list
├── stat_list
│   ├── stat_list
│   │   ├── decl_int
│   │   │   ├── a
│   │   │   └── decl_int
│   │   │       └── b
│   │   └── assign
│   │       ├── a
│   │       └── 5
│   └── assign
│       ├── b
│       └── 0
├── for
│   ├── init
│   │   ├── assign
│   │   │   ├── a
│   │   │   └── 0
│   │   └── cond
│   │       ├── <
│   │       ├── a
│   │       ├── 5
│   │       ├── assign
│   │       │   ├── a
│   │       │   ├── +
│   │       │   └── a
│   │       └── 1
│   ├── if_else
│   │   ├── if
│   │   │   ├── ==
│   │   │   ├── a
│   │   │   ├── 2
│   │   │   ├── 0
│   │   │   ├── assign
│   │   │   │   ├── b
│   │   │   │   ├── +
│   │   │   │   └── a
│   │   │   └── assign
│   │   │       ├── +
│   │   │       ├── b
│   │   │       └── 1
│   └── while
│       ├── >
│       ├── b
│       ├── 0
│       └── assign
│           ├── b
│           ├── -
│           └── b
└── 1
```

Task 5 - Intermediate Code Generation

Input:

```
task5_icg > C input.c
1  int a;
2  int b;
3  a = 5;
4  b = 0;
5  for (a = 0; a < 5; a = a + 1) {
6      if (a % 2 == 0) {
7          b = b + a;
8      } else {
9          b = b + 1;
10     }
11 }
12 while (b > 0) {
13     b = b - 1;
14 }
15
```

Output:

```
swagm@ubuntu:~/Documents/Mini_Compiler_Project/task4_opt$ cd ../task5_icg
swagm@ubuntu:~/Documents/Mini_Compiler_Project/task5_icg$ lex tac.l
swagm@ubuntu:~/Documents/Mini_Compiler_Project/task5_icg$ yacc -d tac.y
swagm@ubuntu:~/Documents/Mini_Compiler_Project/task5_icg$ gcc lex.yy.c y.tab.c -lfl -o tac
swagm@ubuntu:~/Documents/Mini_Compiler_Project/task5_icg$ ./tac < input.c

=== Three Address Code ===
a = 0
b = 0
a = 5
b = 0
t0 = a < 5
t1 = a + 1
t2 = a % 2
t3 = t2 == 0
t4 = b + a
b = t4
t5 = b + 1
b = t5
if t3 == 0 goto L0
goto L1
L0:
L1:
a = 0
L2:
if t0 == 0 goto L3
a = t1
goto L2
L3:
t6 = b > 0
t7 = b - 1
b = t7
L4:
if t6 == 0 goto L5
goto L4
L5:

=== Quadruple Table ===
Op      Arg1      Arg2      Result
-----
=        0        -         a
=        0        -         b
=        5        -         a
=        0        -         b
<        a        5         t0
+        a        1         t1
%        a        2         t2
==       t2        0         t3
+        b        a         t4
=        t4        -         b
+        b        1         t5
=        t5        -         b
if==0    t3        -         L0
goto     -        -         L1
label    -        -         L0
label    -        -         L1
=        0        -         a
label    -        -         L2
if==0    t0        -         L3
=        t1        -         a
goto     -        -         L2
label    -        -         L3
>        b        0         t6
-        b        1         t7
=        t7        -         b
label    -        -         L4
if==0    t6        -         L5
goto     -        -         L4
label    -        -         L5
```

Task 6 - Code Optimization

Input:

```
task6_optimization > input.txt
1  a = 0
2  b = 0
3  a = 5
4  b = 0
5  t0 = a < 5
6  t1 = a + 1
7  t2 = a % 2
8  t3 = t2 == 0
9  t4 = b + a
10 b = t4
11 t5 = b + 1
12 b = t5
13 if t3 == 0 goto L0
14 goto L1
15 L0:
16 L1:
17 a = 0
18 L2:
19 if t0 == 0 goto L3
20 a = t1
21 goto L2
22 L3:
23 t6 = b > 0
24 t7 = b - 1
25 b = t7
26 L4:
27 if t6 == 0 goto L5
28 goto L4
29 L5:
30
```

Output:

```
• swapna@ubuntu:~/Documents/Mini_Compiler_Project/task5_icg$ cd ../task6_optimization
• swapna@ubuntu:~/Documents/Mini_Compiler_Project/task6_optimization$ python3 icg_optimize.py input.txt --print

--- Optimized TAC ---
a = 0
b = 0
a = 5
b = 0
b = 5
b = 6
if t3 == 0 goto L0
goto L1
L0:
L1:
a = 0
L2:
if t0 == 0 goto L3
a = 6
goto L2
L3:
b = 5
L4:
if t6 == 0 goto L5
goto L4
L5:
```

Task 7 - Assembly Code Generation

Input:

```
task7_assembly > input22.txt
1  a = 0
2  b = 0
3  a = 5
4  b = 0
5  b = 5
6  b = 6
7  if t3 == 0 goto L0
8  goto L1
9  L0:
10 L1:
11 a = 0
12 L2:
13 if t0 == 0 goto L3
14 a = 6
15 goto L2
16 L3:
17 b = 5
18 L4:
19 if t6 == 0 goto L5
20 goto L4
21 L5:
```

Output:

```
• swapna@ubuntu:~/Documents/Mini_Compiler_Project/task6_optimization$ cd ../task7_assembly
• swapna@ubuntu:~/Documents/Mini_Compiler_Project/task7_assembly$ python3 assemble.py input22.txt

--- Optimized ARM Assembly Output ---

LDR r0, =a    @ load a
MOV r0, #0
LDR r1, =b    @ load b
MOV r1, #0
MOV r0, #5
MOV r1, #5
MOV r1, #6
LDR r2, =t3   @ load temporary t3
CMP r2, #0
BEQ L0
B L1
L0:
L1:
MOV r0, #0
L2:
LDR r3, =t0   @ load temporary t0
CMP r3, #0
BEQ L3
MOV r0, #6
B L2
L3:
MOV r1, #5
L4:
LDR r4, =t6   @ load temporary t6
CMP r4, #0
BEQ L5
B L4
L5:

Assembly code also saved in input22.s
```

All Output: file: run_and_show.sh

```
• swapna@ubuntu:~/Documents/Mini_Compiler_Project$ cd ~/Documents/Mini_Compiler_Project
• swapna@ubuntu:~/Documents/Mini_Compiler_Project$ chmod +x run_and_show.sh
• swapna@ubuntu:~/Documents/Mini_Compiler_Project$ ./run_and_show.sh
  🚀 Starting Mini Compiler full run...
    • Running Task 1 (Lexer)...

=====
🔧 Task 1 - Lexer
File: /home/swapna/Documents/Mini_Compiler_Project/outputs/task1_lexer_output.txt
=====
1 int --> KEYWORD

1 a --> IDENTIFIER
1 ; --> PUNCTUATION
2 int --> KEYWORD
2 b --> IDENTIFIER
2 ; --> PUNCTUATION
3 a --> IDENTIFIER
3 = --> ASSIGN_OP
3 5 --> CONST
3 ; --> PUNCTUATION
4 b --> IDENTIFIER
4 = --> ASSIGN_OP
4 0 --> CONST
4 ; --> PUNCTUATION
5 for --> KEYWORD
5 ( --> PARENTHESIS
5 a --> IDENTIFIER
5 = --> ASSIGN_OP
```

```
5 0 --> CONST
5 ; --> PUNCTUATION
5 a --> IDENTIFIER
5 < --> REL_OP
5 5 --> CONST
5 ; --> PUNCTUATION
5 a --> IDENTIFIER
5 = --> ASSIGN_OP
5 a --> IDENTIFIER
5 + --> ARITH_OP
5 1 --> CONST
5 ) --> PARENTHESIS
5 { --> PARENTHESIS
6 if --> KEYWORD
6 ( --> PARENTHESIS
6 a --> IDENTIFIER
6 % --> ARITH_OP
6 2 --> CONST
6 == --> REL_OP
6 0 --> CONST
6 ) --> PARENTHESIS
6 { --> PARENTHESIS
7 b --> IDENTIFIER
7 = --> ASSIGN_OP
7 b --> IDENTIFIER
7 + --> ARITH_OP
7 a --> IDENTIFIER
7 ; --> PUNCTUATION
8 } --> PARENTHESIS
8 else --> KEYWORD
8 { --> PARENTHESIS
9 b --> IDENTIFIER
9 = --> ASSIGN_OP
9 b --> IDENTIFIER
9 + --> ARITH_OP
9 1 --> CONST
9 ; --> PUNCTUATION
10 } --> PARENTHESIS
11 } --> PARENTHESIS
12 while --> KEYWORD
12 ( --> PARENTHESIS
12 b --> IDENTIFIER
12 > --> REL_OP
12 0 --> CONST
```

```

12 ) --> PARENTHESIS
12 { --> PARENTHESIS
13 b --> IDENTIFIER
13 = --> ASSIGN_OP
13 b --> IDENTIFIER
13 - --> ARITH_OP
13 1 --> CONST
13 ; --> PUNCTUATION
14 } --> PARENTHESIS

```

Press Enter to continue to next output...

• Running Task 2 (Symbol Table)...

Task 2 – Symbol Table

File: /home/swapna/Documents/Mini_Compiler_Project/outputs/task2_symbol_table_output.txt

TOKEN#	DATA TYPE	TOKEN TYPE	TOKEN VALUE	LINES	DIM	ADDRESS
1	int	KEYWORD	int	1, 2	1	9999999
2	int	IDENTIFIER	a	1, 3, 5, 6, 7	1	9999999
3	int	IDENTIFIER	b	2, 4, 7, 9, 12, 13	1	9999999
4	for	KEYWORD	for	5	1	9999999
5	if	KEYWORD	if	6	1	9999999
6	else	KEYWORD	else	8	1	9999999
7	while	KEYWORD	while	12	1	9999999

Press Enter to continue to next output...

• Running Task 3 (Syntax Analysis)...

syntax.y: warning: 37 shift/reduce conflicts [-Wconflicts-sr]

syntax.y: note: rerun with option '-Wcounterexamples' to generate conflict counterexamples

Task 3 – Syntax Analysis

File: /home/swapna/Documents/Mini_Compiler_Project/outputs/task3_syntax_output.txt

```

decl:int
id:a
semicolon
decl:int
id:b
semicolon
id:a
assignop:=
num:5
semicolon
id:b
assignop:=
num:0
semicolon
for
(
id:a
assignop:=
num:0
semicolon
id:a
compop:<
num:5
semicolon
id:a
assignop:=
id:a
addop:+
num:1
)
{
if
(
id:a
modop:%
num:2
compop:=
num:0
)
{
id:b
assignop:=
id:b
addop:+
id:a

```

```

semicolon
}
else
{
id:b
assignop:=
id:b
addop:+
num:1
semicolon
}
}
while
(
id:b
compop:>
num:0
)
{
id:b
assignop:=
id:b
subop:-
num:1
semicolon
}

```

```

Symbol Table
Symbol Scope dtype Value
a 1 int 0
b 1 int 0

```

Press Enter to continue to next output...

```

* Running Task 4 (AST)...
tree.y: warning: 25 shift/reduce conflicts [-Mconflicts-sr]
tree.y: note: rerun with option '-Mcounterexamples' to generate conflict counterexamples

```

```

# Task 4 - AST
File: /home/swagga/Documents/Mini_Compiler_Project/outputs/task4_ast_output.txt

```

```

Parsing and building AST...

```

```

stmt_list
├── stmt_list
│   ├── stmt_list
│   │   ├── decl_int
│   │   │   ├── a
│   │   │   └── decl_int
│   │   │       └── b
│   │   └── assign
│   │       ├── a
│   │       └── 5
│   └── assign
│       ├── b
│       └── 0
├── for
│   ├── init
│   │   ├── assign
│   │   │   ├── a
│   │   │   └── 0
│   │   └── cond
│   │       ├── <
│   │       │   ├── a
│   │       │   └── 5
│   │       └── assign
│   │           ├── a
│   │           └── +
│   │               └── a
│   │                   └── 1
│   └── if_else
│       ├── if
│       │   ├── ==
│       │   │   ├── a
│       │   │   └── 2
│       │   └── assign
│       │       ├── b
│       │       └── +
│       │           ├── b
│       │           └── a
│       └── assign
│           ├── b
│           └── +
│               ├── b
│               └── 1
└── while
    ├── >
    │   ├── b
    │   └── 0
    └── assign
        ├── b
        └── -
            └── b
                └── 1

```

```

Press Enter to continue to next output...

```

```

* Running Task 5 (Intermediate Code Generation - ICG)...

```

```

# Task 5 - Intermediate Code Generation (ICG)
File: /home/swagga/Documents/Mini_Compiler_Project/outputs/task5_icg_output.txt

```

```

=== Three Address Code ===

```

```

a = 0
b = 0
a = 5
b = 0
t0 = a < 5
t1 = a + 1
t2 = a % 2
t3 = t2 == 0
t4 = b + a
b = t4
t5 = b + 1
b = t5
if t3 == 0 goto L0
goto L1
L0:
a = 0
L2:
if t0 == 0 goto L3
a = t1
goto L2
L1:
t6 = b > 0
t7 = b - 1
b = t7
L4:
if t5 == 0 goto L5
goto L4
L5:

```

```

=== Quadruple Table ===

```

Op	Arg1	Arg2	Result
=	0	-	a
=	0	-	b
=	5	-	a
=	0	-	b
<	a	5	t0
+	a	1	t1
%	a	2	t2
==	t2	0	t3
+	b	a	t4
=	t4	-	b
+	b	1	t5
=	t5	-	b
if==0	t3	-	L0
goto	-	-	L1
label	-	-	L0
label	-	-	L1
=	0	-	a
label	-	-	L2
if==0	t0	-	L3
=	t1	-	a
goto	-	-	L2
label	-	-	L3
>	b	0	t6
=	b	1	t7
label	-	-	L4
if==0	t6	-	L5
goto	-	-	L4
label	-	-	L5

```

Press Enter to continue to next output...

```

• Running Task 6 (Optimization)...

Task 6 – Optimization

File: /home/swapna/Documents/Mini_Compiler_Project/outputs/task6_optimization_output.txt

--- Optimized TAC ---

```
a = 0
b = 0
a = 5
b = 0
b = 5
b = 6
if t3 == 0 goto L0
goto L1
L0:
L1:
a = 0
L2:
if t0 == 0 goto L3
a = 6
goto L2
L3:
b = 5
L4:
if t6 == 0 goto L5
goto L4
L5:
```

Press Enter to continue to next output...

• Running Task 7 (Assembly)...

Task 7 – Assembly

File: /home/swapna/Documents/Mini_Compiler_Project/outputs/task7_assembly_output.txt

--- Optimized ARM Assembly Output ---

```
LDR r0, =a    @ load a
MOV r0, #0
LDR r1, =b    @ load b
MOV r1, #0
MOV r0, #5
MOV r1, #5
MOV r1, #6
LDR r2, =t3   @ load temporary t3
CMP r2, #0
BEQ L0
B L1
L0:
L1:
MOV r0, #0
L2:
LDR r3, =t0   @ load temporary t0
CMP r3, #0
BEQ L3
MOV r0, #6
B L2
L3:
MOV r1, #5
L4:
LDR r4, =t6   @ load temporary t6
CMP r4, #0
BEQ L5
B L4
L5:
```

Assembly code also saved in input22.s

Press Enter to continue to next output...

🎉 All tasks completed. Outputs are saved in: /home/swapna/Documents/Mini_Compiler_Project/outputs
./run_and_show.sh: line 121: /bin: Is a directory

Conclusion:

This Mini Compiler Project successfully demonstrates the key concepts of compiler design. By implementing all compiler phases using Flex, Bison, GCC, and Python, it provides practical insights into language translation and optimization techniques. Future improvements could include richer semantic analysis, more aggressive optimizations, register allocation via graph coloring, and better error recovery mechanisms.